Explainability in Reinforcement Learning

Maxime Alaarabiou

Monday 24th August, 2026

Compiled on 24/8/2026 at 22:01

Chapter 1

Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Network (NN). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial NN.

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

1.1 Historical Developments

NN and DL emerged from an ambition to study the expressive capabilities of organic brain. They can be seen as a particularly striking example of biomimicry, as they draw inspiration from living systems to address scientific challenges. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [Rosenblatt, 1958]. It was designed to adjust its parameters from input–output pairs in order to approximate a target function. However, its fundamental limitations were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or (XOR) [Minsky and Papert, 1969]. At the same time, it was demonstrated that stacking multiple perceptrons could, in principle, represent such functions. However, no effective training procedure for these multi-layer architectures was available at the time. It was only in 1986 that researchers introduced gradient descent as an effective method for training

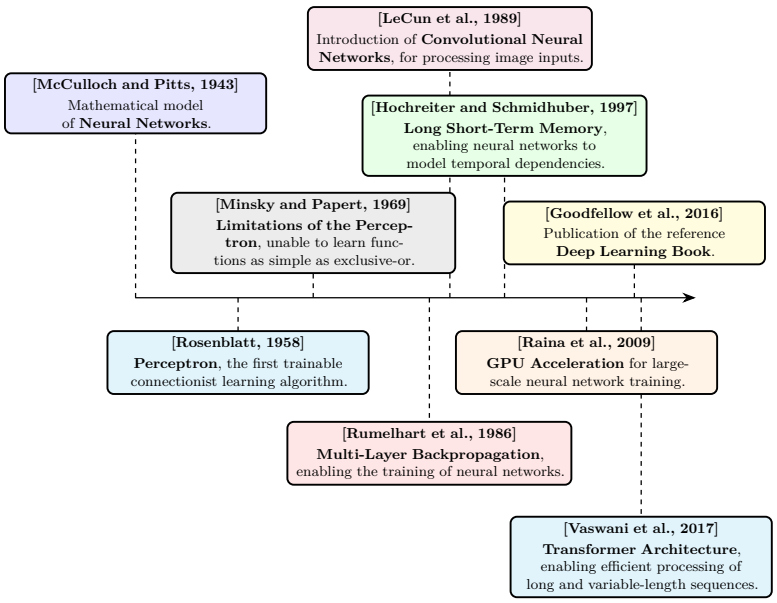

Figure 1.1: Evolution of deep learning.

deep NN [Rumelhart et al., 1986]. From this point onward, the emergence of DL is retrospectively situated, even though the term itself appeared later. DL is defined as the training of deep NN using gradient-based optimization methods. For simplicity, the term NN will refer throughout this manuscript to deep NN.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. At the same time, progress in hardware, particularly the use of graphics processing units, greatly increased computational capacity and made it feasible to train large-scale models [Krizhevsky et al., 2012].

These changes paved the way for the rise of DL as we know it today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on Large Language Model (LLM) [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once

considered automatable.

Figure 1.1 provides a timeline of the key milestones that shaped deep learning.

1.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. The range of problems that can be addressed with DL is extremely broad. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are progressively updated during training so as to make it a better approximation of the target function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive parameter updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."
> [Mitchell, 1997]

When this formulation is adapted to DL, the following correspondences hold:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

The function to be approximated is denoted by f. As with any function, it defines a mapping from an input to an output: $x \in \mathcal{X}$ denotes an input and $y \in \mathcal{Y}$ the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces throughout this manuscript. This assumption is adopted solely to facilitate the presentation, as the framework can be readily extended to more general settings. The notation $\hat{\cdot}$ indicates that a given object is an approximation of another. Since the goal of the trained NN is to approximate f, this approximation is

denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, only a finite set of observations generated by f is available, organized in what is called a dataset. To distinguish between the different samples, an index i is introduced, allowing input–output pairs of the form $(x^{(i)}, y^{(i)})$ to be defined. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}. \tag{1.1}$$

Now that the way the target function f is represented has been made explicit, one can ask what it means for a function $\hat{f}$ to be a good approximation of this function. It means that, for a given input $x^{(i)}$, the NN output $\hat{y}^{(i)}$ is close to the true output $y^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is introduced that measures the discrepancy between predictions and target values. In DL, this function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the Mean Squared Error (MSE), defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \tag{1.2}$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}). \tag{1.3}$$

This formulation captures the mathematical core of the learning objective, namely finding a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. Since the target function f may take many different forms, $\hat{f}$ must be expressive enough to approximate a broad range of such functions. NN architectures are well suited to this requirement, as they can represent a wide range of functional mappings, whose composition is detailed in the following section.

1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation, the focus is placed on the MultiLayer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of DL. Indeed, all the essential mechanisms of DL are present in it, making it a particularly suitable framework for introducing key concepts.

In this section, we therefore begin by examining the artificial neuron, the fundamental computational unit underlying the MLP. We then explain how

these neurons are organized into layers. Finally, we conclude with a general description of the neural network as a whole.

1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation a is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right), \tag{1.4}$$

where:

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w (relative to the others) indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological

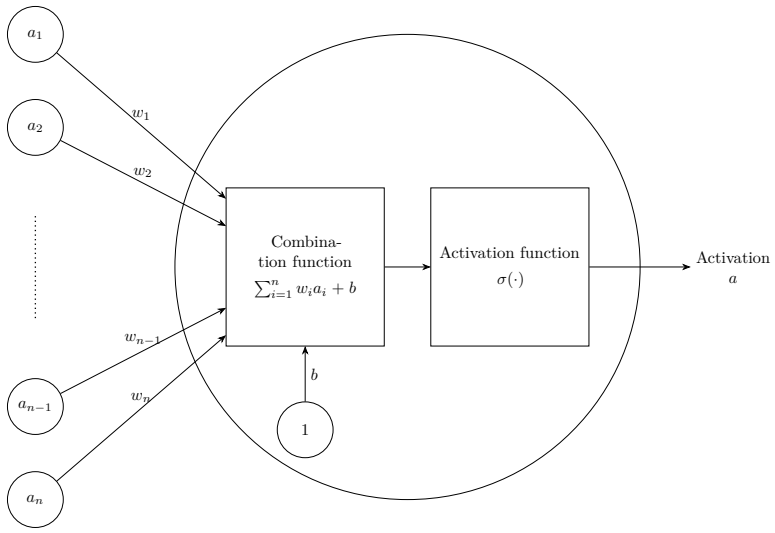

Figure 1.2: Artificiale neurone.

neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. As a result, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers (also referred to as intermediate layers), which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer: its neurons do not compute activations but instead directly encode the components of the input vector x. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions

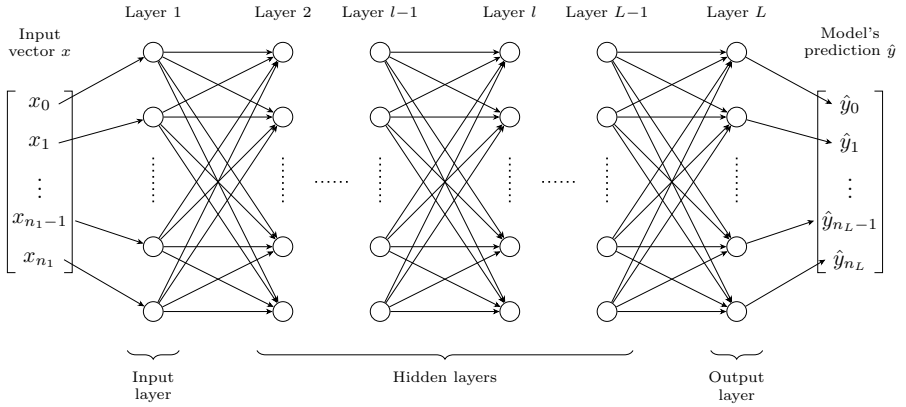

Figure 1.3: Multilayer Perceptron Architecture.

between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \ldots, a_{n_l}^{(l)}]. \tag{1.5}$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \tag{1.6}$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions, and in practice, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NNs [Telgarsky, 2016]. A common interpretation is that each layer acts as a processing step, so that more complex target functions require more such steps to be well approximated, although this interpretation remains grounded in empirical observation rather than formal

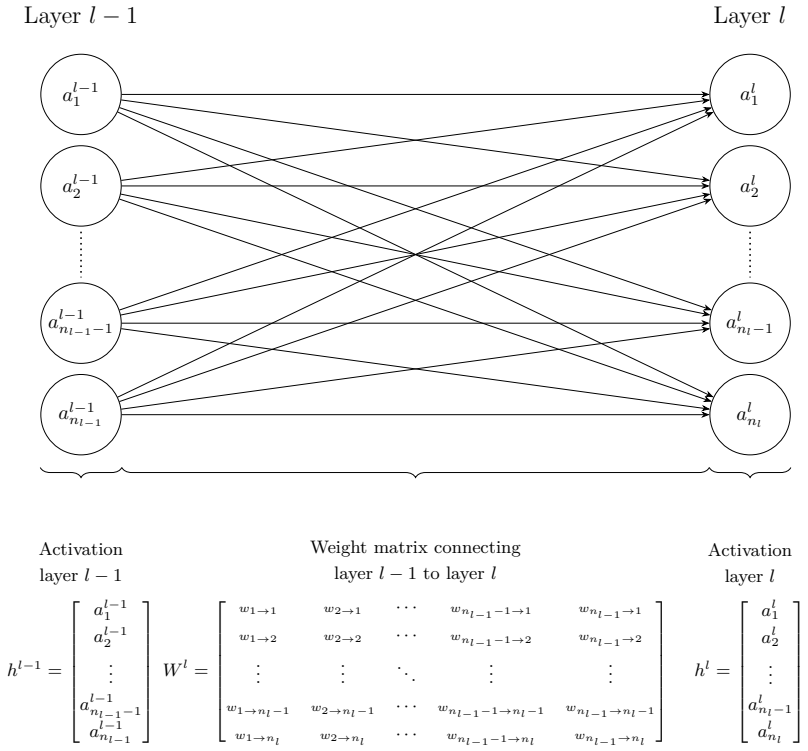

Figure 1.4: Interaction between layers.

theoretical justification. An entire chapter of this manuscript is devoted to studying the information that can be extracted from the activations of these intermediate layers (Chapter **??**).

1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed L):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}. \tag{1.7}$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \le l \le L. \end{cases} \tag{1.8}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}. \tag{1.9}$$

In this manuscript, θ is considered as a vector in $\mathbb{R}^P$, where P denotes the total number of parameters, such that $\theta = [w_1, \ldots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, can enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to θ, which controls the strength of the connections between neurons. The following sections examine how these parameters can be adjusted to obtain a better approximation of the target function.

1.4 Learning

Now that the structure of a NN has been detailed, the next step is to examine how its parameters are adjusted to accomplish the assigned task. This brings us to the core of DL, namely the learning process itself. As a reminder, the objective of training a NN is to adjust its parameters so that the network provides a good approximation of the target function. Mathematically, this objective is expressed as:

$$\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}). \tag{1.10}$$

However, the NN $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, the optimum cannot be determined analytically by directly studying the properties of the function. Instead, approximate optimization methods are required to find suitable parameter values for NNs. The class of algorithms used for this optimization is known as gradient descent methods. Numerous variants of gradient descent have been developed for NN training, each designed to improve specific aspects of the optimization process. For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

This section is organized into three steps. The first derives the gradient of the loss using the chain rule. The second shows how this gradient is used to update the network parameters. The third discusses the convergence properties of the resulting algorithm.

1.4.1 Computing the Gradient

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. We denote this initial state by $\theta^{(1)}$. The goal is to iteratively update this vector so that the network gets better at approximating the target function. To do this, we need to quantify the influence of each parameter w_k on the loss $\mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$, which is given by the partial derivative $\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_k}$.

However, w_k can belong to any layer of the network. Computing this derivative therefore requires propagating the effect of a change in w_k through all the layers that follow it, up to the loss. To do so, we consider each layer as a function of several variables, whose inputs are collected in the vector $\mathbf{h}^{(l-1)}$ and whose outputs are collected in the vector $\mathbf{h}^{(l)}$. Since these quantities are functions of several variables, we rely on the Jacobian matrix, the generalization of the derivative to this setting. The Jacobian matrix quantifies how a small change in each input component affects each output component. For a function $\mathbf{f}\colon \mathbb{R}^n \to \mathbb{R}^m$, we denote by $J_{\mathbf{f}}(\mathbf{x})$ the Jacobian of $\mathbf{f}$ evaluated at $\mathbf{x}$, which collects the partial derivatives of every output component with respect to every input component:

$$J_{\mathbf{f}}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}. \tag{1.11}$$

Each entry tells us how one output component reacts to a small change in one input component.

To propagate the effect of w_k through the successive layers, we need the derivative of a composition of several functions. For a composition of differentiable functions $\mathbf{f} \circ \mathbf{g}$, the chain rule states that its Jacobian, evaluated at $\mathbf{x}$, is the product of the Jacobians of each function, each evaluated at the appropriate point,

$$J_{\mathbf{f}\circ\mathbf{g}}(\mathbf{x}) = J_{\mathbf{f}}(\mathbf{g}(\mathbf{x}))\, J_{\mathbf{g}}(\mathbf{x}). \tag{1.12}$$

This is the tool required to determine the impact of w_k on the output of the network, regardless of where it is located. A network computes its output through a sequence of layer activations $\mathbf{h}^{(1)}, \ldots, \mathbf{h}^{(L)}$, with each layer depending only on the one before it, and with $\mathbf{h}^{(L)} = \hat{f}_\theta(x^{(i)})$. As a result, a parameter w_k belonging to layer l affects the loss $\mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})$ through a chain of dependencies, it first changes $\mathbf{h}^{(l)}$, which changes $\mathbf{h}^{(l+1)}$, and so on until the output layer $\mathbf{h}^{(L)}$, from which the loss is computed. Since $x^{(i)}$ and $y^{(i)}$ are fixed for a given example, the loss can be viewed as a function of w_k alone, and we seek the effect of a small change in w_k on this value. Applying the chain rule across this chain gives

$$\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_k} = J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})\, J_{\mathbf{h}^{(L)}}(\mathbf{h}^{(L-1)}) \cdots J_{\mathbf{h}^{(l+1)}}(\mathbf{h}^{(l)})\, J_{\mathbf{h}^{(l)}}(w_k), \tag{1.13}$$

where:

- $J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})$ denotes the Jacobian of $\mathcal{L}$ with respect to its first argument, evaluated at $(\mathbf{h}^{(L)}, y^{(i)})$,
- each subsequent $J_{\mathbf{h}^{(j+1)}}(\mathbf{h}^{(j)})$ denotes the Jacobian of layer $j+1$ with respect to its input $\mathbf{h}^{(j)}$,
- $J_{\mathbf{h}^{(l)}}(w_k)$ denotes the Jacobian of layer l with respect to the parameter w_k.

Each factor is therefore the Jacobian of one layer with respect to its input. Multiplying these local Jacobians together propagates the effect of w_k all the way to the loss. This layer-by-layer propagation is what we call backpropagation.

Once we can compute $\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}),y^{(i)})}{\partial w_k}$ for every parameter, we stack all P of these values into one vector, called the gradient of the loss with respect to the parameters:

$$\nabla_\theta \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}) = \left[\frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)})}{\partial w_P} \right]. \tag{1.14}$$

Each component answers the same question for a different parameter, namely, if we slightly increase this parameter, does the loss go up or down, and by how much. A positive value means increasing the parameter increases the loss, whereas a negative value means the opposite. The gradient thus tells us, for every parameter, which direction to move in order to reduce the loss.

1.4.2 Update rule

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads us to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta^{(n)}} \mathcal{L}(\hat{f}_{\theta^{(n)}}(x^{(i)}), y^{(i)}), \tag{1.15}$$

where $\eta > 0$ is the learning rate, which controls the step size.

This parameter η must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta_1, \theta_2, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss. Algorithm 1 summarizes this whole training procedure.

To develop an intuition for this update rule, Figure **??** illustrates it applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum. The point θ_1 represents the initial parameter values, with w_1 and w_2 randomly initialized, and the subsequent points, from θ_2 to θ_6, represent the successive updates produced by applying Equation 1.15, progressively moving the trajectory toward the minimum of the loss function.

1.4.3 Convergence Properties

Having established how SGD updates the parameters, a natural question is whether this procedure is guaranteed to converge. It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, and no such guarantee holds. Figure **??** illustrates the effect of non-convex functions, different starting points lead to distinct local minima.

Despite the lack of theoretical guarantees of convergence to a global optimum in this setting, the simple procedure of random initialization followed by iterative

Algorithm 1: Stochastic Gradient Descent

Input:
Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$
Architecture $\hat{f}$
Loss function $\mathcal{L}$
Learning rate $\eta > 0$
Initial parameters θ (optional)
Output:
Trained parameters θ

---

if θ *is given* **then**
  $\theta^{(1)} \leftarrow \theta$;
else
  Initialize $\theta^{(1)}$ randomly;
$n \leftarrow 1$;
while *stopping criterion not satisfied* **do**
  Select a random example $(x^{(i)}, y^{(i)}) \in \mathcal{D}$;
  Model prediction for the selected example
  $\hat{y}^{(i)} \leftarrow \hat{f}_{\theta^{(n)}}(x^{(i)})$;
  Compute the error between the prediction and the ground truth
  $\ell \leftarrow \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$;
  Compute the gradient of the error since the loss function is differentiable
  $g \leftarrow \nabla_{\theta^{(n)}} \ell$;
  Error minimization via a gradient descent step
  $\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta\, g$;
  $n \leftarrow n + 1$;
Return $\theta^{(n)}$

gradient-based updates is sufficient in practice to train highly complex NN. This empirical success is often attributed to the expressive power of NNs, even though a complete theoretical understanding of this phenomenon remains an open question.

1.5 Evaluation

Now that the process of training a model has been described, the question of how to evaluate it must be addressed. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset

$\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\ell}_{\text{train}}$, is studied. This is an instance of the performance measure P defined in Equation (1.3), restricted to $\mathcal{D}_{\text{train}}$:

$$\bar{\ell}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{train}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}). \tag{1.16}$$

If $\bar{\ell}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate, and the initialization strategy. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. We must now study its ability to generalize. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\ell}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{test}}} \mathcal{L}(\hat{f}_\theta(x^{(i)}), y^{(i)}). \tag{1.17}$$

If $\bar{\ell}_{\text{test}} \gg \bar{\ell}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\ell}_{\text{test}} \approx \bar{\ell}_{\text{train}}$, this indicates that the model generalizes well beyond the training data [Kawaguchi et al., 2022]. The model can therefore be used with confidence for the task on which it was trained.

The ability of a neural network to generalize provides evidence that the model has learned more than a simple memorization of the training examples. Instead, it must construct internal representations that capture regularities present in the data. These representations emerge progressively through the hidden layers during training. Understanding their structure is therefore a key step toward understanding how neural networks generalize. The collection of

these representations is referred as the Latent Space (LS). The next chapter is devoted to the study of these LSs.

Acronyms

DL Deep Learning 1–4, 6, 7, 9, 13

LLM Large Language Model 2
LS Latent Space 14

MLP MultiLayer Perceptron 4, 6
MSE Mean Squared Error 4

NN Neural Network 1–7, 9, 11, 12

SGD Stochastic Gradient Descent 9, 11

Compiled on 24/8/2026 at 22:01

Bibliography

[Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

[Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world's technological capacity to store, communicate, and compute information. science, 332(6025):60–65.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8):1735–1780.

[Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366.

[Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589.

[Kawaguchi et al., 2022] Kawaguchi, K., Bengio, Y., and Kaelbling, L. (2022). Generalization in Deep Learning, page 112–148. Cambridge University Press.

[Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

[LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4):541–551.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.

[Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.

[Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.

[Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In Icml, volume 9, pages 873–880.

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In Conference on learning theory, pages 1517–1539. PMLR.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.